Approach 1 :-

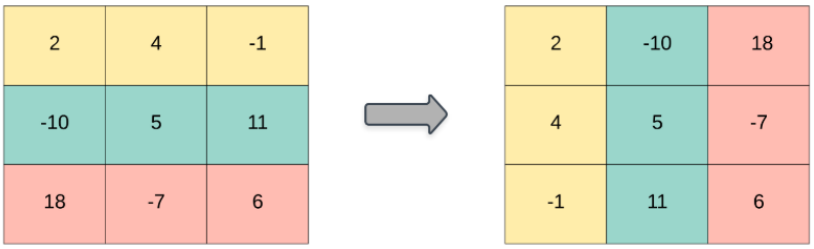
```java
public int diagonalSum(int[][] mat) {
    int sum = 0;
    for(int i=0;i<mat.length;i++){
        for(int j=0;j<mat[0].length;j++){
            if(i==j || i+j == mat.length-1){
                sum += mat[i][j];
            }
        }
    }
    return sum;
}
```

Approach 2 :-

```java
public int diagonalSum(int[][] mat) {
    int sum = 0;
    int i = 0;
    int j = mat.length - 1;
    for (int k = 0; k < mat.length; k++) {
        if (i != j) {
            sum += mat[k][i] + mat[k][j];
        } else {
            sum += mat[k][i];
        }
        i++;
        j--;
    }
    return sum;
}
```

---

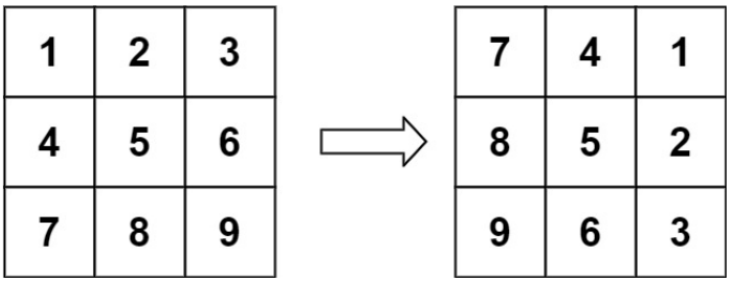
867. Transpose Matrix



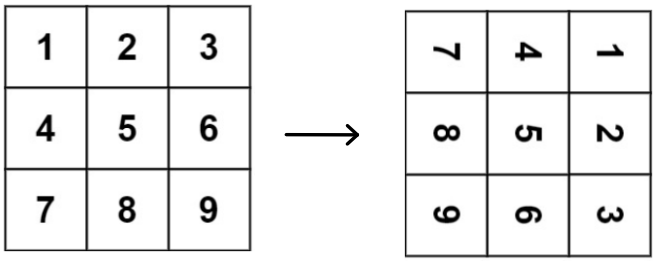
Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [[1,4,7],[2,5,8],[3,6,9]]

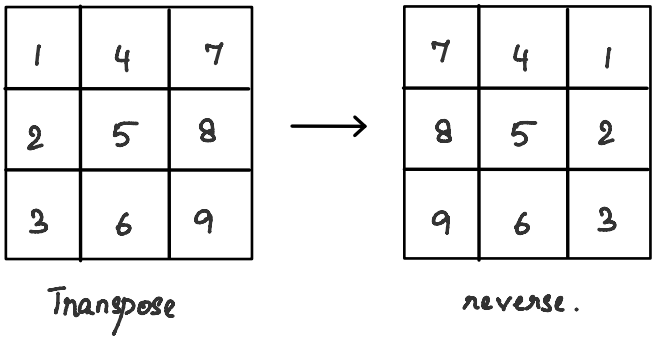
```java
public int[][] transpose(int[][] matrix) {
    int n = matrix.length;
    int m = matrix[0].length;
    int ans[][] = new int[m][n];
    for(int i=0;i<m;i++){
        for(int j=0;j<n;j++){
            ans[i][j] = matrix[j][i];
        }
    }
    return ans;
}
```

48. Rotate Image



Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [[7,4,1],[8,5,2],[9,6,3]]



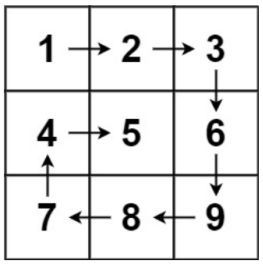



```
void Transpose(int arr[][]) {
    for (int i = 0; i < arr.length; i++) {
        for (int j = i + 1; j < arr[0].length; j++) {
            int temp = arr[i][j];
            arr[i][j] = arr[j][i];
            arr[j][i] = temp;
        }
    }
}
```

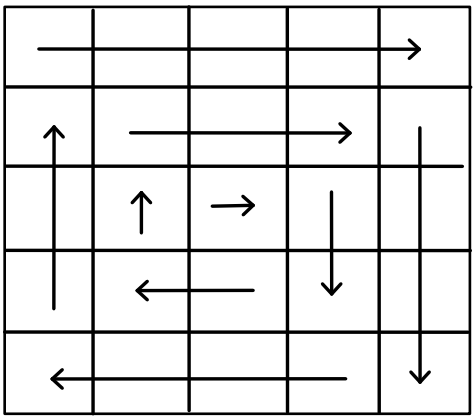
```
void Reverse(int arr[]) {
    int i = 0, j = arr.length - 1;
    while (i < j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
        i++;
        j--;
    }
}
```

```
public void rotate(int[][] matrix) {
    Transpose(matrix);
    for (int i = 0; i < matrix.length; i++) {
        Reverse(matrix[i]);
    }
}
```

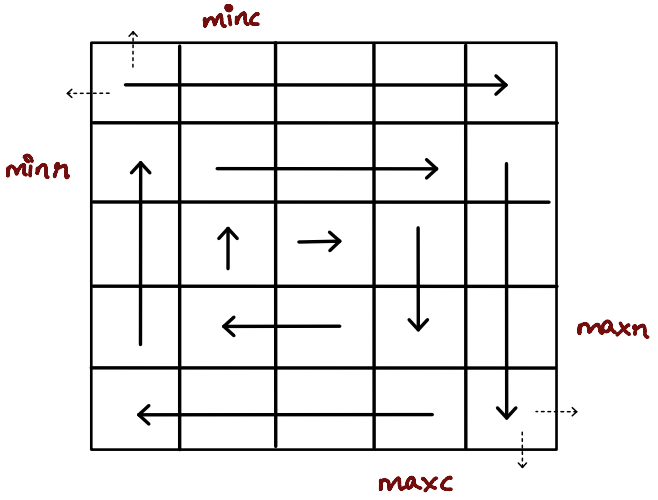
54. Spiral Matrix



Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [1,2,3,6,9,8,7,4,5]



total = n × c;




```
while ( cnt < total){                    && cnt < total
for (int i = minc ; i<= maxc; i++) {
        val = arr[minn][i];
        al. add (val);
          cnt ++;
    }
minn++;
for ( int  i = minn; i<= maxn; i++) {
        val = arr[i][maxc];
        al. add (val);
          cnt ++;
    }
maxc - -;
for(int i = maxn; i >= minn; i - -){
        val = arr [i][minc];
        al . add( val);
          cnt ++;
    }
minc ++;
for(int i = maxc ; i >= minc; i--){
        val = arr[maxn] [i];
        al. add (val);
          cnt ++;
    }
maxn--;
```

```java
public List<Integer> spiralOrder(int[][] matrix) {
    List<Integer> al = new ArrayList<>();
    int r = matrix.length, c = matrix[0].length;
    int minr = 0, minc = 0, maxr = r - 1, maxc = c - 1;
    int cnt = 0, total = r * c;
    while (cnt < total) {
        for (int i = minc; i <= maxc && cnt < total; i++) {
            al.add(matrix[minr][i]);
            cnt++;
        }
        minr++;
        for (int i = minr; i <= maxr && cnt < total; i++) {
            al.add(matrix[i][maxc]);
            cnt++;
        }
        maxc--;
        for (int i = maxc; i >= minc && cnt < total; i--) {
            al.add(matrix[maxr][i]);
            cnt++;
        }
        maxr--;
        for (int i = maxr; i >= minr && cnt < total; i--) {
            al.add(matrix[i][minc]);
            cnt++;
        }
        minc++;
    }
    return al;
}
```

---

121. Best Time to Buy and Sell Stock

Input: prices = [7,1,5,3,6,4]
Output: 5
Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5.
Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.

```java
public int maxProfit(int[] prices) {
    int max = 0;
    int buy = prices[0];
    for(int i=1;i<prices.length;i++){
        if(prices[i]<buy){
            buy = prices[i];
        }else{
            int profit = prices[i]-buy;
            max = Math.max(max,profit);
        }
    }
    return max;
}
```

41. First Missing Positive

Input: nums = [1,2,0]
Output: 3
Explanation: The numbers in the range [1,2] are all in the array.

positive Number starts from 1.



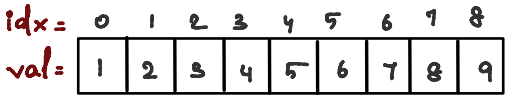
op: 8



op: 10 → len = 9 = 9+1 = 10.



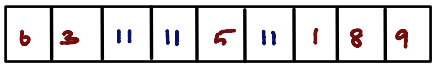
op: 2.



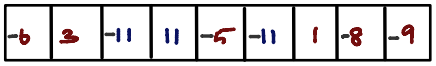
[1 to n+1]



convert Neg values into impossible range.

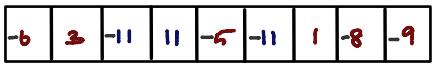


n+2.



val = 5.

arr[val-1]



arr[i] > 0
i+1

if all val are Neg.
n+1

```java
public int firstMissingPositive(int[] nums) {
    int n = nums.length;
    for (int i = 0; i < n; i++) {
        if (nums[i] <= 0) {
            nums[i] = n + 2;
        }
    }
    for (int i = 0; i < nums.length; i++) {
        int val = Math.abs(nums[i]);//5
        if (val > 0 && val <= n) {
            nums[val - 1] = Math.abs(nums[val - 1]) * -1;
        }
    }
    for (int i = 0; i < n; i++) {
        if (nums[i] > 0) {
            return i + 1;
        }
    }
    return n + 1;
}
```